

Constraint-Based Problem Solving – Assessment Tool 2

Course: Artificial Intelligence (CSA17)

Question 1: Doctor – Shift Assignment (Backtracking Search)

Variables: D1, D2, D3 **Domain (each):** {Morning, Afternoon, Night} **Order used for the check**
D2<D3: Morning=1, Afternoon=2, Night=3

Constraints: (i) D1 ≠ Night. (ii) D2 < D3 (using the Morning<Afternoon<Night order). (iii) All-different – one doctor per shift. (iv) D3 ≠ Morning. (v) Every doctor gets exactly one shift.

Backtracking trace (variable order D1 → D2 → D3, value order Morning → Afternoon → Night)

Step	Assignment attempted	Constraint check	Result
1	D1 = Morning	(i) D1 ≠ Night → satisfied (Morning)	Accept, go to D2
2	D2 = Morning	(iii) All-different → conflicts with D1	Reject, try next value
3	D2 = Afternoon	(iii) All-different → OK (D1=Morning, D2=Afternoon)	Accept, go to D3
4	D3 = Morning	(iv) D3 ≠ Morning → violated	Reject, try next value
5	D3 = Afternoon	(iii) All-different → conflicts with D2	Reject, try next value
6	D3 = Night	(iv) D3 ≠ Morning → OK. (i) D1 ≠ Night → OK (D1=Morning). (ii) D2 < D3 → Afternoon(2) < Night(3) → OK. (iii) all-different → OK	Accept – all constraints satisfied

No further backtracking is required – every variable (D1, D2, D3) has been assigned a shift and all five constraints are satisfied at once the first time all three variables are consistently assigned.

Final valid schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

(Note: a second valid schedule also exists – D1 = Afternoon, D2 = Morning, D3 = Night – since the constraints do not uniquely force D1's value. Backtracking with the value order Morning → Afternoon → Night returns the schedule above as the first solution found.)

Question 2: Robot Path Planning in a 5×5 Grid

Grid setup used for this solution (rows 0–4 top to bottom, columns 0–4 left to right):

- Start S = (0,0), Goal G = (4,4)
- Obstacles (blocked cells): (1,1), (2,1), (3,1) – a vertical wall in column 1, open only at row 0 and row 4
- Moves allowed: Up, Down, Left, Right | Cost of every move = 1
- Heuristic: Manhattan Distance, $h(n) = |\text{row}_G - \text{row}_n| + |\text{col}_G - \text{col}_n|$

Note on the algorithm: since a priority queue and a heuristic are both specified, the frontier is expanded in **Best-First order using $f(n) = g(n) + h(n)$** (this is the standard way to combine a priority queue with Manhattan distance, and it guarantees the optimal path – unlike plain Breadth-First Search, which ignores cost and heuristic entirely). The trace below shows node expansions and priority-queue (frontier) updates.

Step-by-step node expansion and priority queue (frontier) updates

It.	Node expanded (g, h, f)	New nodes generated & added to frontier
1	(0,0) g=0 h=8 f=8	(0,1) f=8, (1,0) f=8
2	(0,1) g=1 h=7 f=8	(0,2) f=8 [(1,1) is an obstacle – skipped]
3	(1,0) g=1 h=7 f=8	(2,0) f=8 [(1,1) is an obstacle – skipped]
4	(0,2) g=2 h=6 f=8	(0,3) f=8, (1,2) f=8
5	(2,0) g=2 h=6 f=8	(3,0) f=8 [(2,1) is an obstacle – skipped]
6	(0,3) g=3 h=5 f=8	(0,4) f=8, (1,3) f=8
7	(1,2) g=3 h=5 f=8	(2,2) f=8
8	(3,0) g=3 h=5 f=8	(4,0) f=8 [(3,1) is an obstacle – skipped]
9	(0,4) g=4 h=4 f=8	(1,4) f=8
10	(1,3) g=4 h=4 f=8	(2,3) f=8
11	(2,2) g=4 h=4 f=8	(3,2) f=8
12	(4,0) g=4 h=4 f=8	(4,1) f=8
13	(1,4) g=5 h=3 f=8	(2,4) f=8
14	(2,3) g=5 h=3 f=8	(3,3) f=8
15	(3,2) g=5 h=3 f=8	(4,2) f=8

16	(2,4) g=6 h=2 f=8	(3,4) f=8
17	(3,3) g=6 h=2 f=8	(4,3) f=8
18	(3,4) g=7 h=1 f=8	(4,4) g=8 h=0 f=8 → GOAL

Note: because the Manhattan heuristic is admissible and consistent, and the vertical wall of obstacles still leaves a clear path with the same length as the unobstructed Manhattan distance, $f(n)$ stays constant at 8 for every node on a shortest path – so the search effectively expands outward in a breadth-first wavefront until the goal is reached, which is why the same optimal cost is found regardless of whether the algorithm is described as Best-First Search or BFS.

Optimal path and cost

Tracing parent pointers back from the goal gives:

(0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4)

Total path cost = 8 (8 moves × cost 1 each), which equals the Manhattan distance from S to G, confirming this is the optimal path even with the obstacles present.

Question 3: Autonomous Rescue Robot in a Disaster Building

a) Effect of constraints on the robot's decision-making

- **4-directional movement** fixes the branching factor of the search tree to at most 4 actions per state, which the robot must evaluate at every room/cell.
- **Blocked cells** mean the robot must verify feasibility of a move before/while executing it – invalid moves are pruned from the action set at that state.
- **Limited sensing (adjacent cells only)** makes the environment partially observable to the robot: it cannot plan a complete path in advance because it does not know the full map, only what is immediately next to it.
- **Cost 1 (normal) vs. cost 2 (risky zone)** turns the grid into a weighted graph, so the robot cannot just minimize the number of moves – it must minimize cumulative weighted cost, favouring longer-but-safer routes over shorter-but-riskier ones when the total cost is lower.
- **Dynamic knowledge updates (online search)** mean the robot must revise its internal map and re-plan every time it senses something new, rather than computing one fixed plan at the start.

b) Solution approach: an Online Search Agent (Online/LRTA*-style search built on Uniform Cost principles)

Because the map is not known in advance, classical offline algorithms (plain BFS, UCS, A*) cannot be run to completion before the robot moves – they all assume the full state space is known. The appropriate approach is an **online search agent** that interleaves sensing, computation, and acting:

- **Step 1 – Sense:** at the current cell, observe the four adjacent cells and record which are open, blocked, or risky.
- **Step 2 – Update knowledge:** add the newly sensed information to the robot's internal map (known-cost graph); unexplored cells keep a default heuristic estimate (Manhattan distance to G).
- **Step 3 – Select action:** among the known/sensed neighbours, choose the one minimizing the estimated cost-to-goal, $g(\text{known-so-far}) + h(\text{Manhattan estimate})$ – the same principle as Uniform Cost Search / A*, but applied only over the locally known graph.
- **Step 4 – Act and move:** execute the chosen move; if it leads to a dead end (all neighbours blocked or already visited with no better option), backtrack to the previous cell, as in online depth-first search.
- **Step 5 – Learn:** if the actual cost of a cell differs from its previous estimate (e.g., it turns out to be a risky zone costing 2), update that cell's stored cost so future decisions/re-traversals improve – this is the core idea behind Learning Real-Time A* (LRTA*).

- **Step 6 – Repeat** Steps 1–5 until the goal cell G (the survivor's location) is reached; the total cost is the sum of the actual move costs along the path actually travelled.

c) Demonstration of AI concepts

- **Intelligent agent:** the robot follows the classic sense → decide → act loop – it perceives adjacent-cell states through its sensors, decides the next move using its search strategy, and acts through its movement actuators.
- **Rationality:** the robot is rational, not omniscient – given only the percepts and knowledge it has accumulated so far, it selects the action that is expected to minimize cost while avoiding known blocked or unnecessarily risky cells. It cannot guarantee the global optimum in advance, but it acts optimally with respect to the information available at each decision point.
- **Environment type:** partially observable (only adjacent cells are sensed), discrete (grid of rooms), sequential (each move affects which moves are possible next), and effectively static apart from the robot's own actions (obstacles do not move) – but unknown, since the full map is not given in advance and must be discovered incrementally.

d) Justification for the chosen approach

An online search agent (built on Uniform-Cost/A*-style cost minimization) is the most suitable approach because it is specifically designed for exactly this situation: a partially observable environment where the full state space is unavailable beforehand. Offline algorithms such as plain BFS, UCS, or A* all require the complete graph (all rooms, connections, and costs) to be known before search begins – something the rescue robot simply does not have, since it can only sense adjacent cells. By interleaving sensing, replanning, and action, the online approach lets the robot safely discover the building layout while it moves, correctly account for the extra cost of risky zones as they are discovered, backtrack sensibly when it hits dead ends, and still converge toward a low-cost path to the survivor – something none of the purely offline search strategies could do under these constraints.